

Map Website — Sprint 1 PRD, Revision 2

Pedro Silva · Rob Hefferon · AI Builder Space Proseminar Revision 2, 7 October 2026. Supersedes [SilvaHefferon_ProjectBrief.pdf](#) (1 October 2026).

Revision 1 is unchanged on disk and still the record of what we set out to build. This document records what we have since decided, discovered, or had to change. Where the two disagree, this one is current.

What changed since Revision 1

Area	Revision 1	Now
Boston vs. global	Open question	Decided: global-capable code, per-city curated data
Restaurant quality	"Established sources, not star ratings alone"	A weighted scoring model over 9 source types
Restaurant data source	Unspecified	Google Places (New) + Yelp, both paid
Bathroom data	Risk: user-updated apps are incomplete	Coverage is fine; attributes are the gap
Out-and-back routes	Listed beside loops	Not an API feature — custom code
Recognition data	Not mentioned	Hand-curated per city; currently all placeholders
Cost	Not mentioned	Needs a Google billing account

Two of these can derail the sprint: the **billing dependency** and the **curation backlog**. Both have their own sections below.

The idea and the user

Unchanged. Map Website is an interactive web map for "foodie runners" — people new to a city who care about both their run and their next meal. It brings custom running routes, public bathrooms, and authentic local restaurants onto one map.

Scope for this sprint

A web map with two working features, **run separately**: route generation and restaurant search.

Must-haves

#	Feature	State
1	Route generator: adjustable distance, start point, loop or out-and-back	UI complete against a stub generator
2	Route elevation	Blocked — see open questions
3	Restaurant finder filtered by price and "recognised" picks	Built
4	Restaurant quality from established sources, not star ratings alone	Built — see the scoring model

Out of scope (unchanged)

- Public bathroom finder — later, as a sub-feature of route planning
- Showing restaurants along or near a generated route
- User accounts; saving or sharing routes
- User-submitted reviews
- Mobile app (stretch goal only)

Scope discipline has held: none of the above has been started. The page has a responsive breakpoint so it is usable on a phone, which is not the same as a mobile app and cost nothing.

Decisions made since Revision 1

Launch area: global code, Boston data

Routing and OpenStreetMap data are worldwide, so restricting the code to Boston would have bought nothing. Cities are configured in `shared/cities.json`; the restaurant API takes `?city=<key>`. What is genuinely Boston-only is the hand-curated recognition data.

Remaining UI work: a city picker. The page uses the default city today.

Routing provider: OpenRouteService, not Google

Google's Directions API cannot generate a loop of a target distance — it answers "start to end," not "start plus a distance." ORS does it in one parameter (`round_trip`), returns an elevation profile, needs no credit card, and permits drawing results on a non-Google basemap.

Consequence not in Revision 1: `round_trip` produces loops only. **Out-and-back is our own code** — pick a bearing, route to a point at half the distance, reverse and append. Budget for it separately.

Restaurant data: Google Places (New) + Yelp

OpenStreetMap was measured first. Within 1.5 km of Boston Common it has 312 restaurants — 99% named, 65% with a cuisine tag, 56% with a website, but only **37% with opening hours**. Good enough to place pins, too thin to tell a runner where to eat now. Hence Google for existence, hours and price, Yelp as a second rating source, and our own curation for quality.

Bathrooms: the risk was mis-stated

Revision 1 assumed public-toilet data would be incomplete because existing apps rely on manual updates.

OpenStreetMap returned 28 public toilets within 1.5 km of Boston Common with no manual effort. The real gap is **attributes**: most are unnamed and many lack **access** and **fee**, so "there is a toilet here" is reliable while "you may use it" often is not. For a mid-run stop, the second question is the feature. Carry this into the bathroom sub-feature.

Restaurant quality: the scoring model

Revision 1 said quality should come from established sources rather than Google stars alone. That is now a defined model. Weights live in `restaurants/scoring.py` as named constants.

Ingredient	Points
Michelin star / Bib Gourmand / Guide	10 / 8 / 6
Eater 38 / Eater Heatmap	8 / 4
City or tourism-board food guide	4 each, max 2
TheFork, Zomato	2
Local food creator list	2 each, max 3
Each extra independent publisher agreeing	+2
Google rating, review-adjusted, above 4.4	up to 3
Yelp rating, review-adjusted, above 4.0	up to 3

Rules worth stating at PRD level, because they are judgment calls:

- **Ratings cannot outrank critics.** Google and Yelp together cap at 6 — below a single Michelin star or an Eater 38 place. This keeps the brief's intent while still using rating data.
- **Ratings are adjusted for review count.** 4.9 from 14 reviews scores nothing; 4.6 from 2,000 scores well. Under 50 reviews never scores.
- **One publisher counts once.** A Michelin star and a Michelin Guide mention do not both score.
- **Chains are removed** via an editable blocklist.

- **"Recognised"** means at least one curated source vouches for the place. The tab shows only recognised places by default.

Each result shows its score breakdown. That is designed for the success test: when the food expert disagrees, the breakdown says which weight to change.

Terminology: Revision 1's "quintessential local" is implemented as **"recognised"** plus a numeric **"local score."** Use those terms from here.

Cost and dependencies (new)

Revision 1 did not mention cost. It is now a real constraint.

- **Google Places (New) requires a billing account** — a card on file before the first call.
- **Yelp Fusion is paid** after a free trial. It is optional; either key alone works.
- **OpenRouteService is free** and needs no card.

Mitigations already built: every Google and Yelp response is cached in `.cache/` for 7 days by default (capped at 30, the limit Google's terms allow for most Places content); search points are rounded to ~100 m so nearby searches share a cache entry; changing filters never calls an API — only moving the point or distance does. A cold search costs at most 3 Google and 2 Yelp requests. `RESTAURANTS MOCK=1` forces demo mode.

Demo mode runs the whole UI with no keys and no spend, using fictional fixtures behind a banner.

Open: nobody has set a monthly spend cap or agreed who pays.

Curation: the critical path (new)

Michelin, Eater, TheFork, Zomato, city guides and creator lists have no usable public API, and most of their terms forbid scraping. Recognition is therefore a **hand-written file per city**: `data/recognition/boston.json`.

That file currently contains 13 entries, all placeholders for fictional restaurants. With real API keys and no real entries, nothing is recognised.

This is unstated work on the critical path of our own success test: the expert cannot review a recognised list until the list exists. Entries must be copied from the published source with a link, never written from memory, and each carries the edition or date because lists go stale.

Owner and deadline: unassigned. This is the single most schedule-critical gap in the sprint.

How it is built (new)

Not in Revision 1, but it constrains how we work.

- **One map, owned by shared/**. Features never create a map; they add layers through a contract (`view.addLayer({ id, type, data, paint, legend })`). Layer ids are namespaced `routes:` / `restaurants:` and the shared code rejects anything else, because MapLibre ids are global.

- **One local server** (`python server/app.py`, standard library only). It serves the site and each feature's API from one process.
- **API keys live in `.env`**, read only on the server, never in client JavaScript — the repo is public. `.env` is gitignored and has never been committed.
- **Features plug in without editing shared files.** A feature adds endpoints by writing `<feature>/api.py` with a `register(router)` function, and map layers via its own `layers.js`. Both are already wired up.

Success criteria

Unchanged in substance, with one gap now explicit.

1. **Restaurants.** Map Website generates a recommendation list and a local expert reviews it. Success is approval of **at least 90%** of the list.
2. **Routes.** A test runner runs a route the site generated and confirms it works as planned.

Gap: "90% of the list" still has no list length. 90% of 10 and 90% of 100 are different bars. Fix before the expert sees anything.

Blocked on: the expert and the test runner are still unrecruited, and criterion 1 additionally depends on the curation backlog above.

Open questions and risks

#	Question	Status
1	A permanent name to replace "Map Website"	Open, unchanged
2	Recruit the local food expert	Open, unchanged
3	Recruit the test runner	Open, unchanged
4	Is elevation an input or a display ?	Open — now blocking
5	Who enters the recognition data, and by when?	New, critical
6	Monthly API spend cap, and who pays	New
7	How many results does the expert review?	New
8	Boston vs. global	Closed — global code, Boston data

#	Question	Status
9	Bathroom data completeness	Closed — coverage fine, attributes thin

On question 4: ORS has no "give me a hilly route" parameter. If elevation is a *display* ("show me the climb"), it is nearly free. If it is a *control* ("give me a flat 5 k"), it means generating several candidates and selecting by ascent — a materially bigger feature. Revision 1 lists it beside adjustable distance, which implies a control. Decide before the ORS work starts.

Status at this revision

Branch	Ahead of <code>main</code>	State
<code>routes</code>	4 commits	Full UI against a stub generator; no real routing yet
<code>restaurants</code>	6 commits	Search, ranking, filters, map pins, tests

Neither feature is merged. **The integration checkpoint — both features drawing on one map — has not happened**, and both branches have grown considerably since they diverged. Doing that merge is the next step, ahead of new feature work.

Route generation is otherwise blocked only on an OpenRouteService key and a `routes/api.py` endpoint; the client already calls a single `generateRoute()` function built to be swapped from stub to server call without touching the surrounding UI.